



**合肥工业大学
计算机与信息学院**

**计算机科学与技术专业新生研讨课
课程报告**

姓 名： 戴成雪

专业班级： 计算机科学与技术 22-3 班

学 号： 2022212603

2024 年 1 月 1 日

目录

一、	NumPy 简介	3
1.1	什么是 NumPy	3
1.2	NumPy 的组成	3
二、	NumPy 安装	3
三、	NumPy 的基础操作	4
3.1	创建数组	4
3.2	通用函数	7
四、	基于 NumPy 的 ID3 算法	9
4.1	什么是 ID3 算法	9
4.2	利用 ID3 算法思想画出决策树	10
4.3	用 python 计算信息熵	13
4.4	用 python 返回最大信息熵的行列	13
4.5	切割分支节点	14
4.6	递归构建决策树	14
4.7	输出决策树	14
4.8	对输入的值进行结果的预测	15
4.9	输入学习的数据	15
4.10	调用算法	16
4.11	预测数据	16
五、	写在最后	17
5.1	心得体会	17
5.2	参考资料	17

一、 NumPy 简介

1.1 什么是 NumPy

NumPy 是 Python 中科学计算的基础包。它是一个 Python 库，提供多维数组对象，各种派生对象（如掩码数组和矩阵），以及用于数组快速操作的各种 API，其核心是 `ndarray` 对象，它封装了 python 原生的同数据类型的 n 维数组，为了保证其性能优良，其中有许多操作都是代码在本地进行编译后执行的。

NumPy 数组有助于对大量数据进行高级数学和其他类型的操作。通常，这些操作的执行效率更高，比使用 Python 原生数组的代码更少与此同时，NumPy 还提供了大量的数学函数库，包括线性代数函数、傅立叶变换以及复数处理函数等。

Numpy 库是机器学习四个基础库（Numpy、Matplotlib、Pandas、Sklearn）之一，机器学习算法中大部分都是调用 Numpy 库来完成基础数值计算的，它是大量 Python 数学和科学计算包的基础。

由此可见，NumPy 对于处理大型矩阵运算、统计分析、信号处理、图像处理等方面具有关键作用，是现代数据科学技术栈的重要组成部分。

1.2 NumPy 的组成

• N 维数组对象(ndarray)

这是 NumPy 的核心特性，可以处理多维度的数组与矩阵运算。NumPy 数组在创建时具有固定的大小，更改 `ndarray` 的大小将创建一个新数组并删除原来的数组。NumPy 数组中的元素都需要具有相同的数据类型，因此在内存中的大小相同。

`ndarray` 由两部分构成：

1. 实际的数据
2. 描述这些数据的元数据

• 大量计算复杂函数的函数库

NumPy 提供了大量的数学函数库，包括线性代数函数、傅立叶变换以及复数处理等。对数组元素进行逐个处理的函数，例如算术运算（加法、减法、乘法、除法等）、数学函数（平方根、指数、对数等）、三角函数以及更多其他数学运算。提供了一系列数学运算和逻辑运算函数，这些函数可以应用于整个数组，无需使用循环即可实现向量化计算。

• 特殊的切片机制

类似 Python 列表的索引和切片机制，但扩展到多维数组。此外，还支持更复杂的索引技术，如布尔索引和花式索引。

二、 NumPy 安装

本实验平台为 Python，可以先在 python 平台 www.python.org/downloads 下载 python，安装时：Add python.exe to Path 需要选勾。在安装 Python 的过程中，pip 通常会自动安装在系统中。接着我们利用 pip 来下载 NumPy 库

打开 Windows 命令行，输入指令：

`pip install numpy` 下载 NumPy 库

显示下图即为安装成功。

```
Microsoft Windows [版本 10.0.22621.3007]
(c) Microsoft Corporation. 保留所有权利。

C:\Users\lenovo>pip install numpy
Collecting numpy
  Obtaining dependency information for numpy from https://files.pythonhosted.org/packages/ad/11/52f8e97fd84c91105b651d25a122f8deed6d3519afb14f9771fac1c9b7de/numpy-1.26.3-cp312-cp312-win_amd64.whl.metadata
  Downloading numpy-1.26.3-cp312-cp312-win_amd64.whl.metadata (61 kB)
    61.2/61.2 kB 364.1 kB/s eta 0:00:00
  Downloading numpy-1.26.3-cp312-cp312-win_amd64.whl (15.5 MB)
    15.5/15.5 MB 5.4 MB/s eta 0:00:00
Installing collected packages: numpy
Successfully installed numpy-1.26.3

[notice] A new release of pip is available: 23.2.1 -> 23.3.2
[notice] To update, run: python.exe -m pip install --upgrade pip

C:\Users\lenovo>
C:\Users\lenovo>
```

接下来直接在 python 中导入 numpy 库即可调用库中的内容：

```
import numpy
```

调用时：arr=numpy.array([1, 2, 3, 4, 5])

```
import numpy
arr=numpy.array([1,2,3,4,5])
print(arr)
[1 2 3 4 5]
```

但是通常，为了简化复杂类型签名，在 python 中 numpy 通常以别名 np 导入，即为例使得代码更加简洁易读。

别名：类型别名通过将类型分配给别名来定义。（用 as）。

例如：

```
>>> import numpy as np
>>> arr=numpy.array([1,2,3,4,5])
>>> print(arr)
[1 2 3 4 5]
>>> arr=np.array([1,2,3,4,5])
>>> print(arr)
[1 2 3 4 5]
```

三、 NumPy 的基础操作

3.1 创建数组

Numpy 中有非常多的函数可以用来创建不同类型的数组：

- array():

如“二、NumPy 安装”中的示例，就是用了 array() 函数创建一个 NumPy ndarray 对象。将列表、元组或任何类似数组的对象传递给 array() 方法（即括号内填的类型），可以生成一个 ndarray 对象。如上图，我们将一个列表对象传递给 array()方法，我们用 type()来查看新生成的 arr 的类型：

```
>>> import numpy
>>> arr=numpy.array([1,2,3,4,5])
>>> print(type(arr))
<class 'numpy.ndarray'>
```

由此可以验证，生成的为 `ndarray` 对象。

同理，传递对象为元组之类的均为这个操作，一般使用该方法来创造自定义数组的情况比较多，因为该方法比较自由。

`array(object, dtype=None, *, copy=True, order='K', subok=False, ndmin=0, like=None)`

`object` 接收 `array_like`。表示想要创建的数组。无默认。

`dtype` 接收 `data-type`。表示数组所需的数据类型。如果未给定，则选择保存对象所需的最小类型。默认为 `None`。

`ndmin` 接收 `int`。指定生成数组应该具有的最小维数。默认为 `None`。

• `arange()`

`arange()`的用法时用于在给定的区间内生成均匀间隔的数组。例如：

```
>>> import numpy as np
>>> print(np.arange(10))#0-9的整型，步长为1
[0 1 2 3 4 5 6 7 8 9]
>>> print(np.arange(1,12))#1-11的整型，步长为1
[1 2 3 4 5 6 7 8 9 10 11]

>>> print(np.arange(1,5,1.5))#1-5的浮点数，步长为1.5
[1. 2.5 4.]
>>> print(np.arange(1,4,2.5))#1-4的浮点数，步长为2.5
[1. 3.5]
>>> print(np.arange(10.0))#0-9的浮点数，步长为1
[0. 1. 2. 3. 4. 5. 6. 7. 8. 9.]
```

由上述代码可见，可以生成自定步长、自定范围内的均匀数组。如果所给定的范围过大难以打印时，会自动省略中间部分，如下图所示：

```
>>> print(np.arange(100000000))
[ 0 1 2 ... 99999997 99999998 99999999]
```

同时，区间取值也可以倒着取，步长为负值，例如下图：

```
>>> print(np.arange(10,2,-2))
[10 8 6 4]
```

如果步长取值超过所给定的范围，则会返回一个空的数组，而不是报错：

```
>>> print(np.arange(10,2,2))
[]
```

`arange([start,] stop[, step,], dtype=None, *, like=None)`

`start` 接收开始数字

`stop` 接收结束位置（不包含）

`start` 和 `stop` 都用来控制区间范围

`step` 选取数字的步长，可以取负值，默认值为 1

`dtype` 指定生成数组的数据类型，如果不指定，则会根据输入的 `start` 和 `stop` 值自动推断，默认为 `None`

• `linspace()`

`linspace()`可以用来返回在间隔[开始，停止]上计算的 `num` 个均匀间隔的样本：

```
>>> import numpy
>>> print(numpy.linspace())
Traceback (most recent call last):
  File "<pyshell#1>", line 1, in <module>
    print(numpy.linspace())
TypeError: linspace() missing 2 required positional arguments: 'start' and 'stop'

>>> print(numpy.linspace(50,20))
[50.      49.3877551  48.7755102  48.16326531 47.55102041 46.93877551
 46.32653061 45.71428571 45.10204082 44.48979592 43.87755102 43.26530612
 42.65306122 42.04081633 41.42857143 40.81632653 40.20408163 39.59183673
 38.97959184 38.36734694 37.75510204 37.14285714 36.53061224 35.91836735
 35.30612245 34.69387755 34.08163265 33.46938776 32.85714286 32.24489796
 31.63265306 31.02040816 30.40816327 29.79591837 29.18367347 28.57142857
 27.95918367 27.34693878 26.73469388 26.12244898 25.51020408 24.89795918
 24.28571429 23.67346939 23.06122449 22.44897959 21.83673469 21.2244898
 20.6122449  20.      ]
>>> print(numpy.linspace(10,20,2))
[10. 20.]
```

由上图可得到 `linspace()` 不允许 `start` 和 `end` 的参数值为空,而产生数组的数量可以为空,并且可以进行倒序分组 (由小到大进行分组)

`linspace(start, stop, num=50, endpoint=True, retstep=False, dtype=None, axis=0)`

`start` 指定了区间的起始点

`stop` 指定了区间的终点 (不包含)

`num` 这是要生成的等间距数值的个数, 默认为 50

`endpoint` 如果为 `True`, 生成的数组将包含从 `start` 到 `stop` 的一个端点; 如果为 `False`, 则不包含 `stop` 点, 默认值为 `ture`

`retstep` 如果设置为 `True`, 该函数除了返回生成的数组外, 还会返回两个相邻元素之间的步长, 默认为 `False`

`dtype` 指定生成数组的数据类型, 如果不指定, 则会根据输入的 `start` 和 `stop` 值自动推断, 默认为 `None`

• `eyes()` 与 `diag()`

`eye` 函数创建在对角线上为 1 的数组, 如图所示:

```
>>> print(numpy.eye(3))#默认给3*3的数组的对角线赋值1
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
>>> print(numpy.eye(3,4))#给3*4的数组的对角线赋值1 (从 (1, 1) 开始)
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]]
```

`diag` 函数创建为对角线上的元素赋值, 自动生成对应大小的数组, 如图所示:

```
>>> print(numpy.diag([1,2,3]))
...
[[1 0 0]
 [0 2 0]
 [0 0 3]]
```

`eye(N, M=None, k=0, dtype=<class 'float'>, order='C', *, like=None)`

N 整数，表示要创建的方阵的行数。如果 M 未指定，则默认 M=N，生成一个正方形的单位矩阵。

M 整数，表示列数。如果不为 None，则创建一个 N x M 大小的矩阵，并在主对角线上放置 1，其余元素为 0。默认值为 None

k 整数，表示对角线偏移量。当 k>0 时，单位元素会出现在从主对角线上方第 k 个位置开始的对角线上；当 k<0 时，单位元素会出现在主对角线下方第|k|个位置开始的对角线上，默认为 0

dtype 指定数组元素的数据类型，默认为 float

order 指定数组内存布局，'C'表示行优先，'F'表示列优先，默认为'C'

diag(v, k=0)

v 如果是一维数组，则提取其对角线元素形成新的数组；如果是标量，则创建一个该标量值填充的对角矩阵。

k 整数，表示对角线偏移量。当 k>0 时，单位元素会出现在从主对角线上方第 k 个位置开始的对角线上；当 k<0 时，单位元素会出现在主对角线下方第|k|个位置开始的对角线上，默认为 0

• zeros()和 ones()

使用 zeros 函数创建元素全 0 的数组

使用 ones 函数创建元素全 1 的数组

```
>>> print(np.ones((3,4)))  
[[1. 1. 1. 1.]  
 [1. 1. 1. 1.]  
 [1. 1. 1. 1.]]  
>>> print(np.zeros((3,4)))  
[[0. 0. 0. 0.]  
 [0. 0. 0. 0.]  
 [0. 0. 0. 0.]]
```

注意：输入的对象一定是一个元组(a,b,c,...)内部数量取决于要生成几维数组

zeros(shape, dtype=float, order='C', *, like=None)

ones(shape, dtype=None, order='C', *, like=None)

• 一维数组的生成与二维数组的生成

以以下代码为例：

arr1 = np.array([1, 2, 3, 4]) # 创建一维数组

arr2 = np.array([[1, 2], [3, 4]]) # 创建二维数组（矩阵）

```
>>> import numpy as np  
>>> arr1 = np.array([1, 2, 3, 4]) #创建一维数组  
>>> print(arr1)  
[1 2 3 4]  
>>> arr2 = np.array([[1, 2], [3, 4]]) #创建二维数组  
>>> print(arr2)  
[[1 2]  
 [3 4]]
```

3.2 通用函数

• sum()函数

以以下代码为例：

创建一个一维数组

```

arr1d = np.array([1, 2, 3, 4, 5])
print(np.sum(arr1d))    (计算整个数组的元素和)
>>> arr1d = np.array([1, 2, 3, 4, 5])
>>> print(np.sum(arr1d))
15

# 创建一个二维数组 (矩阵)
arr2d = np.array([[1, 2, 3], [4, 5, 6]])
print(np.sum(arr2d))    (计算所有元素的总和, 即每一行相加再将结果相加)
>>> arr2d = np.array([[1, 2, 3], [4, 5, 6]])
>>> print(np.sum(arr2d))
21

# 沿着指定轴求和
print(np.sum(arr2d, axis=0)) (每一列相加)
# 这表示第一个元素是第一列的和 (1 + 4), 第二个元素是第二列的和 (2 + 5), 以此类推。
>>> print(np.sum(arr2d, axis=0))
[5 7 9]

print(np.sum(arr2d, axis=1)) (每一行相加)
# 这表示第一个元素是第一行的和 (1 + 2 + 3), 第二个元素是第二行的和 (4 + 5 + 6)。
>>> print(np.sum(arr2d, axis=1))
[ 6 15]

```

• 长度函数

虽然没有直接的 `len()` 函数 (这个函数是 Python 内置函数, 用于获取对象长度), 但是我们可以使用 `numpy.shape` 或者 `numpy.size` 来获取数组的长度或元素个数。

```

# 创建一个 numpy 数组
arr = np.array([1, 2, 3, 4, 5])
# 使用 numpy.size 获取数组元素个数, 等同于 len(arr)
print(np.size(arr))
>>> arr = np.array([1, 2, 3, 4, 5])
>>> print(np.size(arr))
5

# 使用 numpy.shape 获取数组的形状, 对于一维数组, 其输出的第一个值即为长度
print(np.shape(arr)[0])
>>> arr = np.array([1, 2, 3, 4, 5])
>>> print(np.shape(arr)[0])
5

```

• unique()函数

对于 `numpy.unique()` 函数, 它用于返回给定数组中的唯一元素, 并自动排序:
以以下例子为例:

```

# 创建包含重复元素的数组
arr = np.array([1, 2, 3, 2, 1, 4, 5, 4])
# 使用 unique 函数找出所有唯一的元素

```



```

unique_elements, counts = np.unique(arr, return_counts=True)
print(unique_elements)
print(counts) #表示各元素在原数组中出现的次数
>>> arr = np.array([1, 2, 3, 2, 1, 4, 5, 4])
>>> unique_elements, counts = np.unique(arr, return_counts=True)
>>> print(unique_elements)
[1 2 3 4 5]
>>> print(counts)
[2 2 1 2 1]

```

• delete()函数

用来删除数组中指定位置的元素或者满足条件的子数组：

```

# 创建一个 numpy 数组
arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
# 删除索引为 1 的行
new_arr = np.delete(arr, 1, axis=0)
print(new_arr) # 输出: [[1 2 3] [7 8 9]]
>>> new_arr = np.delete(arr, 1, axis=0)
>>> print(new_arr)
[[1 2 3]
 [7 8 9]]
# 删除列索引为 1 的列
new_arr = np.delete(arr, 1, axis=1)
print(new_arr)
>>> new_arr = np.delete(arr, 1, axis=1)
>>> print(new_arr)
[[1 3]
 [4 6]
 [7 9]]

```

四、 基于 NumPy 的 ID3 算法

4.1 什么是 ID3 算法

基本的 ID3 算法通过自顶向下构造决策树来进行学习。构造过程是从“那一个属性将在树的根节点进行测试”，即“如何选择最佳的根节点”这个问题开始的。

为了回答这个问题，使用“信息增益”来确定每一个实例属性单独分类训练样例的能力。分类能力最好的属性将被选作树的根节点。然后为根节点属性的每个可能值产生一个分枝，并把训练样例排列到该属性值对应的分枝之下。

然后重复上述整个过程，用每个分支节点关联的训练样例来选取在该点被测试的最佳属性。直到终止条件（没有记录可分或没有属性）得到满足。

那么，信息增益是什么？

信息增益衡量的是通过某个特征划分数据集所减少的“不确定性”或熵。它量化了在进行分类前后的信息变化程度。

它所涉及到的几个量：

1. 熵：

描述了一个随机变量的不确定性或者纯度。对于一个类别标签集合而言，熵越高表示样本类别越分散，不确定性越大。

$$H(D) = - \sum_{i=1}^{n_c} p_i \log_2(p_i)$$

其中，D 是父节点的数据集， p_i 是类别 i 在 D 中出现的概率， n_c 是类别的数量

2. 信息增益：

是指原数据集 D 的熵与根据特征 A 划分后各个子集熵的加权平均值之间的差值。

$$IG(D, A) = H(D) - \sum_{v \in \text{Values}(A)} \frac{|D_v|}{|D|} \cdot H(D_v) \text{ 其中:}$$

- $H(D)$ 是原始数据集的熵。
- A 是当前考虑的特征。
- $\text{Values}(A)$ 是特征 A 可能取的所有值集合。
- D_v 是特征 A 取值为 v 时对应子集的数据集。
- $|D_v|$ 表示子集 D_v 中的样本数。
- $|D|$ 表示原始数据集 D 的样本总数。

因此根据信息增益的大小排序，从而可以取信息增益最大的为根节点，如此反复，知道没有其余的判断选项。

ID3 算法的缺点：

1. ID3 算法倾向于选择具有较多取值的特征进行划分，这样容易导致产生复杂的决策树模型，可能出现过拟合的情况，特别是当训练数据噪声较大时。
2. ID3 算法的特征选择只考虑了离散特征，对于连续特征，需要进行离散化处理才能应用于 ID3 算法。

4.2 利用 ID3 算法思想画出决策树

学习一个人吃饭时是否要等待的决策：

序号	属性						决策： 是否等待
	有否其他 选择	饿否	价格	餐馆类型	餐馆顾客人数	等待时间 (分钟)	
1	是	是	\$\$\$	法式	有人	0~10	是
2	是	是	\$	中餐	客满	30~60	否
3	否	否	\$	快餐	有人	0~10	是
4	是	是	\$	中餐	客满	10~30	是
5	是	否	\$\$\$	法式	客满	> 60	否
6	否	是	\$\$	意大利式	有人	0~10	是
7	否	否	\$	快餐	无人	0~10	否
8	否	是	\$\$	中餐	有人	0~10	是
9	否	否	\$	快餐	客满	> 60	否
10	是	是	\$\$\$	意大利式	客满	10~30	否
11	是	否	\$	中餐	无人	0~10	否
12	否	是	\$	快餐	客满	30~60	是

人工计算方法：

该题中有12个决策，其中6个决策为“是”，6个决策为“否”

① 计算数据集的信息熵：

$$Info(D) = -P(\text{是}) \times \log_2 P(\text{是}) - P(\text{否}) \times \log_2 P(\text{否}) = 1$$

② 计算每个属性的信息增益：

是否有其他选择	决策是	决策否	频率	概率	信息熵
是	2	4	6	$\frac{1}{2}$	$-\frac{1}{2} \log_2 \frac{1}{2} - \frac{2}{2} \log_2 \frac{2}{2}$
否	4	2	6	$\frac{1}{2}$	$-\frac{2}{2} \log_2 \frac{2}{2} - \frac{1}{2} \log_2 \frac{1}{2}$

$$\therefore Info_1(D) = 2 \times (-\frac{1}{2} \log_2 \frac{1}{2} - \frac{2}{2} \log_2 \frac{2}{2}) \times \frac{1}{2} = 0.459$$

$$\therefore \text{信息增益} = 0.541$$

钱否	决策是	决策否	频率	概率	信息熵
是	5	2	7	$\frac{7}{12}$	$-\frac{5}{7} \log_2 \frac{5}{7} - \frac{2}{7} \log_2 \frac{2}{7}$
否	1	4	5	$\frac{5}{12}$	$-\frac{1}{5} \log_2 \frac{1}{5} - \frac{4}{5} \log_2 \frac{4}{5}$

$$\therefore Info_2(D) = (-\frac{5}{7} \log_2 \frac{5}{7} - \frac{2}{7} \log_2 \frac{2}{7}) \times \frac{7}{12} + (-\frac{1}{5} \log_2 \frac{1}{5} - \frac{4}{5} \log_2 \frac{4}{5}) \times \frac{5}{12} = 0.803$$

$$\therefore \text{信息增益} = 0.197$$

价格	决策是	决策否	频率	概率	信息熵
贵	1	2	3	$\frac{3}{12}$	$-\frac{1}{3} \log_2 \frac{1}{3} - \frac{2}{3} \log_2 \frac{2}{3}$
中	2	0	2	$\frac{2}{12}$	$-\frac{2}{2} \log_2 \frac{2}{2} - 0$
便宜	3	4	7	$\frac{7}{12}$	$-\frac{3}{7} \log_2 \frac{3}{7} - \frac{4}{7} \log_2 \frac{4}{7}$

$$Info_3(D) = (-\frac{1}{3} \log_2 \frac{1}{3} - \frac{2}{3} \log_2 \frac{2}{3}) \times \frac{3}{12} + (-\frac{2}{2} \log_2 \frac{2}{2} - 0) \times \frac{2}{12} + (-\frac{3}{7} \log_2 \frac{3}{7} - \frac{4}{7} \log_2 \frac{4}{7}) \times \frac{7}{12} = 0.794$$

$$\therefore \text{信息增益} = 0.205$$

餐馆类型	决策是	决策否	频率	概率	信息熵
法式	1	1	2	$\frac{2}{12}$	$-\frac{1}{2} \log_2 \frac{1}{2} - \frac{1}{2} \log_2 \frac{1}{2}$
中餐	2	2	4	$\frac{4}{12}$	$-\frac{1}{2} \log_2 \frac{1}{2} - \frac{1}{2} \log_2 \frac{1}{2}$
快餐	2	2	4	$\frac{4}{12}$	$-\frac{1}{2} \log_2 \frac{1}{2} - \frac{1}{2} \log_2 \frac{1}{2}$
意大利式	1	1	2	$\frac{2}{12}$	$-\frac{1}{2} \log_2 \frac{1}{2} - \frac{1}{2} \log_2 \frac{1}{2}$

$$\therefore Info_4(D) = 1$$

$$\therefore \text{信息增益} = 0$$

餐馆顾客人数

有人	决策是	决策否	频率	概率	信息熵
有人	4	0	4	$\frac{4}{12}$	0
满座	2	4	6	$\frac{6}{12}$	$-\frac{1}{2} \log_2 \frac{1}{2} - \frac{2}{2} \log_2 \frac{2}{2}$
无人	0	2	2	$\frac{2}{12}$	0

$$Info_5(D) = (-\frac{1}{2} \log_2 \frac{1}{2} - \frac{2}{2} \log_2 \frac{2}{2}) \times \frac{6}{12} = 0.459$$

$$\therefore \text{信息增益} = 1 - 0.459 = 0.541$$

等待时间

0-10	决策是	决策否	频率	概率	信息熵
0-10	4	2	6	$\frac{6}{12}$	$-\frac{2}{3} \log_2 \frac{2}{3} - \frac{1}{3} \log_2 \frac{1}{3}$
10-30	1	1	2	$\frac{2}{12}$	$-\frac{1}{2} \log_2 \frac{1}{2} - \frac{1}{2} \log_2 \frac{1}{2}$
30-60	1	1	2	$\frac{2}{12}$	$-\frac{1}{2} \log_2 \frac{1}{2} - \frac{1}{2} \log_2 \frac{1}{2}$
>60	0	2	2	$\frac{2}{12}$	0

$$Info_6(D) = (-\frac{2}{3} \log_2 \frac{2}{3} - \frac{1}{3} \log_2 \frac{1}{3}) \times \frac{6}{12} + (-\frac{1}{2} \log_2 \frac{1}{2} - \frac{1}{2} \log_2 \frac{1}{2}) \times \frac{2}{12} + 0 = 0.792$$

$$\therefore \text{信息增益} = 0.207$$

由此可得,顾客人数的信息增益max $\rightarrow$ 作为根节点.

顾客人数 是否有其他选择 价格 类型 时间 决策

有人	✓	✓	3★	法	0-10	✓
	X	X	★	快	0-10	✓
	X	✓	2★	意	0-10	✓
	X	✓	2★	中	0-10	✓
无人	✓	X	★	中	0-10	X
	X	X	★	快	0-10	X
客满	X	✓	★	快	30-60	✓
	✓	✓	★	中	10-30	✓
	✓	✓	★	中	30-60	X
	✓	X	3★	法	>60	X
	X	X	★	快	>60	X
	✓	✓	3★	意	10-30	X

由此继续计算客满情况下的信息增益.

是否有其他选择 决策 ✓ 决策 X 频率 概率 信息熵

是	1	3	4	$\frac{4}{6}$	$-\frac{4}{6}\log_2\frac{4}{6} - \frac{2}{6}\log_2\frac{2}{6}$
否	1	1	2	$\frac{2}{6}$	$-\frac{1}{6}\log_2\frac{1}{6} - \frac{1}{6}\log_2\frac{1}{6}$

$$\text{Info}(D') = \frac{4}{6} \times (-\log_2\frac{4}{6} - \log_2\frac{2}{6}) + \frac{2}{6} \times 1 = 0.874$$

信息增益 = 0.126

$$\text{同理可得 } \text{Info}_2(D') = (-\frac{1}{6}\log_2\frac{1}{6} - \frac{1}{6}\log_2\frac{1}{6}) \times \frac{4}{6} + 0 = 0.667 \quad \text{信息增益} = 0.333$$

$$\text{Info}_3(D') = (-\frac{1}{6}\log_2\frac{1}{6} - \frac{1}{6}\log_2\frac{1}{6}) \times \frac{4}{6} + 0 = 0.667 \quad \text{信息增益} = 0.333$$

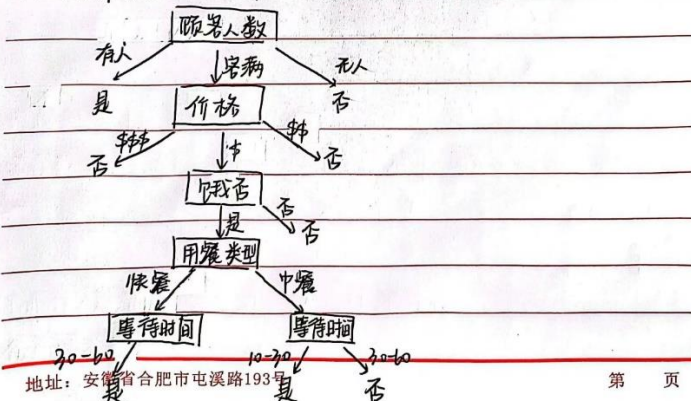
$$\text{Info}_4(D') = \frac{2}{6}(-\frac{1}{6}\log_2\frac{1}{6} - \frac{1}{6}\log_2\frac{1}{6}) + \frac{4}{6}(-\frac{1}{6}\log_2\frac{1}{6} - \frac{1}{6}\log_2\frac{1}{6}) = 0.667 \quad \text{信息增益} = 0.333$$

$$\text{Info}_5(D') = \frac{2}{6}(-\frac{1}{6}\log_2\frac{1}{6} - \frac{1}{6}\log_2\frac{1}{6}) + \frac{4}{6}(-\frac{1}{6}\log_2\frac{1}{6} - \frac{1}{6}\log_2\frac{1}{6}) = 0.667 \quad \text{信息增益} = 0.333$$

后4个信息增益一样, 随便选一个, 这里选择价格类型.

★	饿	快餐	30-60	✓
	饿	中餐	10-30	✓
	饿	中餐	30-60	X
	不饿	快餐	>60	X
3★	饿	法餐	>60	X
	不饿	意餐	10-30	X

得到决策树为:



4.3 用 python 计算信息熵

```
def entropy(labels):
```

```
    # 获取目标变量的唯一值以及统计次数
    unique_labels, label_counts = np.unique(labels, return_counts=True)
    total_count = len(labels)                                # 总样本量个数
    prob = label_counts / total_count                       # 该唯一值的占比
    ent = -np.sum(prob * np.log2(prob))                    # 计算信息熵
    return ent
```

如果出现了无唯一可用的信息熵（信息熵有不只一个最大值），选择出现次数最多的节点：

```
def majority_vote(labels):
```

```
    # 使用 numpy.unique()函数统计每个元素的出现次数
    unique_values, counts = np.unique(labels, return_counts=True)
    majority_label = unique_values[np.argmax(counts)]        # 找到出现次数最多的元素
    return majority_label
```

4.4 用 python 返回最大信息熵的行列

```
def choose_best_feature(data):                                # 选择最佳划分特征,返回索引

    labels = data[:, -1]                                     # 获取当前样本集的目标变量，固定-1表示最后一列是标签
    base_entropy = entropy(labels)                           # 获取当前样本集的信息熵

    best_info_gain = 0.0                                     # 设定初始信息增益为 0
    best_column = -1                                         # 设定初始最优特征列为-1

    for index in range(data.shape[1]-1):                    # 获取当前样本集的每一列
        unique_values = np.unique(data[:, index])            # 获取当前样本集的第 index 列的唯一值

        new_entropy = 0.0                                    # 当前 index 列的条件熵初始设为 0
        sub_data = data[:, [index, -1]]                     # 获取子数据集(仅包含两列)：第 index 列,目标变量

        for value in unique_values:                           # 遍历第 index 列的所有特征值
            # 根据特定的特征值 value 划分出的样本子集，并只获取目标变量这一列
            label_i = sub_data[sub_data[:, 0] == value, 1]
            # 计算 index 列的特征值 value 对应的节点的条件熵，并进行累加
            new_entropy += len(label_i) / len(labels) * entropy(label_i)
        # index 列的信息增益 = 当前样本集的信息熵 - index 列的条件熵
        info_gain = base_entropy - new_entropy

        if info_gain > best_info_gain:                        # 获取信息增益最大对应的 index 列
            best_info_gain = info_gain
            best_column = index

    return best_column                                       # 返回信息增益最大对应的 index 列
```


4.5 切割分支节点

```
def split_data(data, index_column, value_column):    # 给定 index 列, 以及其某个特征值 value
    # 筛选满足条件的样本
    # 获取 index 列中所有特征值为 value 的样本
    filtered_data = data[data[:, index_column] == value_column]
    return np.delete(filtered_data, index_column, axis=1)    # 剔除 index 列, 并返回
```

4.6 递归构建决策树

```
def create_decision_tree(data, columns):
    # data 是 numpy 格式, 包含目标变量
    # 目标变量索引。从 0 开始, 且固定为最后一列
    # columns 为列名称

    labels = data[:, -1]
    # 如果数据集中的所有实例属于同一类别, 则返回该类别
    if len(set(labels)) == 1:
        return labels[0]
    # 如果没有特征可用, 返回实例集中最多的类别
    if data.shape[1] == 1:
        return majority_vote(labels)

    # 获取的是信息增益最大的特征列对应的索引

    best_column_index = choose_best_feature(data)
    bestcolumn = columns[best_column_index]    # 依据索引获取特征列名称
    decision_tree = {bestcolumn: {}}    # 构建当前节点下的决策树
    del(columns[best_column_index])    # 列名称列表剔除已选取的最优特征

    uniqueVals = set(data[:, best_column_index])    # 获取已选取的最优特征的唯一值

    # 依据已选取的最优特征的唯一值, 进行数据切割, 并进行下一节点的树构建
    for value in uniqueVals:
        sub_columns = copy.deepcopy(columns)
        decision_tree[bestcolumn][value] = create_decision_tree(split_data(data,
best_column_index, value), sub_columns)
    return decision_tree
```

4.7 输出决策树

```
def print_decision_tree(decision_tree, indent=""):
    # 遍历并输出决策树

    # 当前节点为叶节点时, 直接输出结果
```

```

if isinstance(decision_tree, str):
    print(indent + decision_tree)
    return

# 当前节点为内部节点时，继续遍历子节点
for key, value in decision_tree.items():
    print(indent + key + ":")
    if isinstance(value, dict):
        print_decision_tree(value, indent + ' ')
    else:
        print(indent + ' ' + value)

```

4.8 对输入的值进行结果的预测

```

def DecisionTree_predict(decision_tree, Columns, test_sample):
    root_feature = list(decision_tree.keys())[0] # 获取根节点的特征
    root_dict = decision_tree[root_feature] # 获取根节点的取值对应的子树

    # 获取根节点特征在特征列表中的索引
    root_feature_index = Columns.index(root_feature)
    for value in root_dict: # 遍历根节点取值对应的子树的所有可能取值
        # 如果测试样本的特征值与当前取值相等
        if test_sample[root_feature_index] == value:
            # 如果该取值对应的子树还是一个字典（非叶子节点）
            if isinstance(root_dict[value], dict):
                # 递归向下查询子树
                class_label = DecisionTree_predict(root_dict[value], Columns, test_sample)
            else: # 如果该取值对应的子树是一个标签（叶子节点）
                class_label = root_dict[value] # 将该标签作为分类结果
    return class_label # 返回分类结果

```

4.9 输入学习的数据

```

def createDataSet(): # 创造示例数据
    dataSet=[['是','是','$$','法式','有人','0-10','是'], 1
              ['是','是','$','中式','客满','30-60','否'], 2
              ['否','否','$','快餐','有人','0-10','是'], 3
              ['是','是','$','中餐','客满','10-30','是'], 4
              ['是','否','$$','法式','客满','>60','否'], 5
              ['否','是','$$','意大利餐','有人','0-10','是'], 6
              ['否','否','$','快餐','无人','0-10','否'], 7
              ['否','是','$$','中餐','有人','0-10','是'], 8
              ['否','否','$','快餐','客满','>60','否'], 9
              ['是','是','$$','意大利餐','客满','10-30','否'], 10
              ['是','否','$','中餐','无人','0-10','否'], 11
              ['否','是','$','快餐','客满','30-60','是']] 12

```

```

        labels = ['是否有其他选择','饿否','价格','餐馆类型','餐馆顾客人数','等待时间',"是否等待"]
        #六个特征+1 个特征列

    return dataSet,labels
data,Columns = createDataSet()
data = np.array(data)

```

4.10 调用算法

```

columns = copy.deepcopy(Columns)
decision_tree = create_decision_tree(data,columns)
print_decision_tree(decision_tree) # 输出决策树结果

```

餐馆顾客人数:

有人:

是

客满:

餐馆类型:

快餐:

饿否:

否:

否

是:

是

中式:

否

法式:

否

中餐:

是

意大利餐:

否

无人:

否

4.11 预测数据

```

testSample=['是','否','$$','中餐','有人','0-10'] # 待测样本
DecisionTree_predict(decision_tree,Columns,testSample)

```

是

五、 写在最后

5.1 心得体会

这是我第一次写实验指导书，在内容上还有非常多的遗漏，因为 NumPy 功能与用法十分丰富的一个库，许多地方的数据处理都有它的影子，而 ID3 仅仅是其中一个比较基础，比较小的机器学习方面的一个小的知识点。

针对于 NumPy 库的函数，还有非常大的填补空间，由于我是想借着 NumPy 库，来实现 ID3 算法，于是仅仅是选取了几个比较经典的、后续 ID3 算法中会用到的一些函数类型。这些其实是远远不够的，NumPy 的数组的使用灵活性远超此。

其实通过这此的实验指导书的整理，我发现其实整理实验指导书是一件非常麻烦的事情，需要消耗大量的精力，因此对之前老师给我们的实验指导书感到更为珍惜。看来真的是只有吃过苦，才能更加品味到糖的甜。

针对这次实验指导书的撰写，我打算继续深入学习 NumPy，继续学习它在其他方面的利用，例如爬虫上的运用，从而能够掌握能多的计算机技巧，更好的提升自己的能力。

5.2 参考资料

在撰写本次实验指导书的过程中，我深深得益于网络上众多宝贵资源的启迪与支持。这些前辈们积累的知识结晶如同灯塔一般，帮助我在探索过程中避开了诸多难题，稳步地完成了这项任务。在此，我整理了一份参考文献清单，对于那些在实验过程中可能遭遇困扰和挑战的后来者，同样可以借鉴以下资料以寻找解决问题的线索和方法。

Numpy 的概念

[Python 数据分析 1: 科学计算工具 Numpy - 知乎 \(zhihu.com\)](https://baike.baidu.com/item/numpy/5678437?fr=ge_al)

https://baike.baidu.com/item/numpy/5678437?fr=ge_al

Numpy 的安装与 Numpy 的基础教程

[python 学习笔记之 numpy 库的使用——超详细 手机插上 u 盘怎么传文件-CSDN 博客](#)

[『行远见大』Python 进阶篇：Numpy 库 - 飞桨 AI Studio 星河社区 \(baidu.com\)](#)

[【Python 入门】NumPy 入门知识介绍，看这一篇足矣 0-d 数组什么意思-CSDN 博客](#)

基于 NumPy 的 ID3 算法：

[基于 NumPy 实现 ID3 决策树算法 numpy 矩阵计算信息增益 ig-CSDN 博客](#)

[基于 Python 实现 ID3 算法 用 python 实现 id3 算法-CSDN 博客](#)

[python 实现决策树分类（ID3） id3 分类算法 python-CSDN 博客](#)

[决策树之 ID3 算法 id3 算法步骤-CSDN 博客](#)

[基于 ID3 算法的决策树分类模型 id3 分类树-CSDN 博客](#)